



Hibernate Framework(part 1)

-by Miss. Geetanjali Gajbhar.

Content:

1. **Introduction**
2. **CRUD using Session Interface's method**
3. **Hibernate/Persistence Life cycle**

Chapter 1

ORM Tool:

- **Object Relational Mapping**
- **Hibernate** is an ORM tool.

Disadvantages of JDBC and advantages of Hibernate.

Sr.No	JDBC	Hibernate
-------	------	-----------

1	A lot of boiler plate code	NO
2	We use SQL in JDBC which is do not support OOPS,	In hibernate we use HQL(Hibernate query language) which support OOPS.
3	Low performance than hibernate	High performance
4	Creating associations between relations is quite hard in JDBC.	Associations like one-to-one, one-to-many, many-to-one, and many-to-many can be acquired easily with the help of annotations.
5	It is a database connectivity tool.	It is a Java framework.
6	JDBC code needs to be written in a try-catch databases block as it throws checked exceptions (SQLException).	Whereas Hibernate manages the exceptions itself by marking them as unchecked.

Hibernate Framework:

- It is an **ORM** tool invented by **Gavin King** in 2001.
- Java framework, ORM tool
- Hibernate provides **methods** which replace conventional SQL queries.
- **Open source**
- **Lightweight**(Support POJO classes)
- **Non-invasive**(i.e. App is loosely coupled hence no need to extends or implements any class/interface).
- Internally Hibernate use **JDBC**.
- We can insert or get the data from the table in the form of **Object**.
- **Hibernate map the java class with table, and object with the table row.**

Mapping:

Java Class → DB Table

Java properties → Table column

Class Object → Table row

For mapping we have 2 annotations in Hibernate

- **@Entity** → To map the class
- **@Id** → to map the Primary key column

There are 3 ways of Hibernate Mapping

1. **XML Approach**
2. **Annotation based approach (Recommended)**
3. **Java Based approach (Recommended)**

Import all annotations from **javax.persistence**

```
import javax.persistence.Entity;  
import javax.persistence.Id;
```

Chapter 2:

CRUD

1. **save()**
 2. **get() or load()**
 3. **update()**
 4. **delete()**
- In hibernate we don't write SQL queries. We have predefined Methods present in **Session Interface**.
 - Hence With the help of Session object we can call these methods.

We are using XML approach

1. Save the object

- Create a new Java Project.
- Add **Hibernate jar** files and **MySQL jar** files.
- How to add the external jar files is explained in JDBC notes.

Create an entity class first:

Note: Maintain Package Structure

package: com.model

```
package com.model;

import javax.persistence.Entity;
import javax.persistence.Id;
/*****
 * @Entity is used to map the class with db table.
 * @Id is used to set the primary key of the table.
 * Java class acts as a table in Db.
 * Every variable defined in the class are the column of the table.
 * Object of the class acts as row of the table.
 *****/
@Entity
public class Student {
    @Id
    private int id;
    private String name;
    private String city;

    public int getId() {
        return id;
    }
}
```

```

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getCity() {
        return city;
    }

    public void setCity(String city) {
        this.city = city;
    }

    public Student(int id, String name, String city) {
        super();
        this.id = id;
        this.name = name;
        this.city = city;
    }

    public Student() {
        super();
    }

}

```

Create Hibernate configuration file at **src level**

hibernate.cfg.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
"-//Hibernate/Hibernate Configuration DTD 3.0//EN"
"http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd" [

<hibernate-configuration>
  <session-factory>

    <!-- Connection properties -->

    <property name="hibernate.connection.driver_class">
com.mysql.jdbc.Driver</property>
    <property name="hibernate.connection.password">root</property>
    <property name="hibernate.connection.url">
jdbc:mysql://localhost:3306/test</property>
    <property name="hibernate.connection.username">root</property>


    <!-- Hibernate properties -->

    <!-- db dialect class -->
    <property name="hibernate.dialect">
org.hibernate.dialect.MySQLDialect</property>

    <!-- to create table -->
    <property name="hbm2ddl.auto">update</property>

    <!-- show sql query -->
    <property name="show_sql">true</property>

    <!-- show sql query in formatted way -->
    <property name="format_sql">true</property>
```

```
<!-- Mapping resources-->

<!-- fully qualified name of pojo class -->
<mapping class="com.model.Student"/>

</session-factory>
</hibernate-configuration>
```

Test class

```
package com.model;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;

public class Test {
    public static void main(String[] args) {

        //configuration
        Configuration cfg=new Configuration()
            .configure("hibernate.cfg.xml");
        SessionFactory sf = cfg.buildSessionFactory();
        Session session = sf.openSession();
        Transaction tx=session.beginTransaction();

        //create 3 objects of Student
        Student s1=new Student(101,"Aman","pune");
        Student s2=new Student(102,"Ajeet","Mumbai");
        Student s3=new Student(103,"Sameer","Delhi");

        //Hibernate methods
        session.save(s1);
        session.save(s2);
```

```

        session.save(s3);
        System.out.println("All data inserted");
        tx.commit();

        //close the session
        session.close();

    }
}

```

Check the output on console.

Check the DB automatically table get created. And the object get saved!

Explanation of above program:

1. First of all we have to **configure** the Hibernate configuration File.
2. Create the configuration file at **src level**
3. By using configuration object we will get **Session Factory object**
4. By using Session Factory object we will get **session object**
5. Session has multiple **predefined methods** to perform CRUD ex. **save, get, load, update ,delete.**

Commonly required Annotations:

@Entity :→Use to make any class an Entity.

@Id →Set as primary key.

@Table →use to change the table details.

@GeneratedValue →We can use this annotation with **@ID** , Because hibernate will automatically. generate values for that using an internal sequence. So don't have to set it manually.

`@Column` → To change the column name, etc. details.

`@Transient` → This tells hibernate not to save this field in DB.

`@Temporal` → It tells hibernate in which format the field needs to be saved. (Ex. Date)

`@LOB` → tells hibernate that it is a large object.

Example:

```
package com.model;

import java.sql.Date;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.Lob;
import javax.persistence.Table;
import javax.persistence.Temporal;
import javax.persistence.TemporalType;
import javax.persistence.Transient;

@Entity
@Table(name="Address_data")
public class Address {
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    @Column(length=20, name="address_id")
    private int id;

    @Column(length=20, name="address_city")
    private String city;

    @Transient
```

```

    private String state;

    @Lob
    private byte[] image;

    @Temporal(TemporalType.DATE)
    private Date addressDate;

    //getter setter
    //Constructor using fields
    //constructor of parent class

}

```

2. Get/Fetch the object

- We have 2 methods to get a single row

1. **get()**
2. **load()**

Using get()

```

package com.model;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;

public class Test_get {
    public static void main(String[] args) {

```

```

//configuration
Configuration cfg=new Configuration()
    .configure("hibernate.cfg.xml");
SessionFactory sf = cfg.buildSessionFactory();
Session session = sf.openSession();

//We dont need Transaction object in get operation
//Transaction tx=session.beginTransaction();

//Hibernate methods
Student st=session.get(Student.class,102);
System.out.println(st.getId());
System.out.println(st.getName());
System.out.println(st.getCity());
System.out.println("Data fetched..!");

//close the seesion
session.close();

}
}

```

Hibernate:

```

select
student0.id as id1_0_0_,
student0.city as city2_0_0_,
student0.name as name3_0_0_
from
Student student0_
where
student0.id=?

```

102

Ajeet
Mumbai
Data fetched..!

Using load()

```
package com.model;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;

public class Test_get {
    public static void main(String[] args) {

        //configuration
        Configuration cfg=new Configuration()
            .configure("hibernate.cfg.xml");
        SessionFactory sf = cfg.buildSessionFactory();
        Session session = sf.openSession();

        //We dont need Transaction object in load operation
        //Transaction tx=session.beginTransaction();

        //Hibernate methods
        Student st=session.load(Student.class,102);
        System.out.println(st.getId());
        System.out.println(st.getName());
        System.out.println(st.getCity());
        System.out.println("Data fetched..!");
    }
}
```

```
//close the session
session.close();

}
}
```

102

Hibernate:

```
select
student0.id as id1_0_0_,
student0.city as city2_0_0_,
student0.name as name3_0_0_
from
Student student0_
where
student0.id=?
```

Ajeet

Mumbai

Data fetched..!

Q. Difference between Get() and load()

get()	load()
If object is not present in DB it will return <u>null</u> .	If object is not present in DB it will return <u>ObjectNotFoundException</u> .
Eager loading: Load every object(hit the DB every time when we call get()). Hence performance is low	lazy loading: Load the object only when getter() methods are called. Until then it returns proxy object(do not hit the DB), hence performance is better.

3. Update operation

- While updating the data, first get the data and then set the updated value.

```
package com.model;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;

public class Test_get {
    public static void main(String[] args) {

        //configuration
        Configuration cfg=new Configuration()
            .configure("hibernate.cfg.xml");
        SessionFactory sf = cfg.buildSessionFactory();
        Session session = sf.openSession();

        Transaction tx=session.beginTransaction();

        //Hibernate methods
        Student st=session.get(Student.class,102);
        System.out.println(st.getId());
        System.out.println(st.getName());
        System.out.println(st.getCity());
        System.out.println("Data fetched..!");

        //update
        st.setCity("Nagpur");
        session.update();
        tx.commit();
        //print updated object
    }
}
```

```

        Student st1=session.get(Student.class,102);
        System.out.println(st.getId());
        System.out.println(st.getName());
        System.out.println(st.getCity());
        System.out.println("Data updated..!");

        //close the session
        session.close();

    }
}

```

Hibernate:

```

select
student0.id as id1_0_0_,
student0.city as city2_0_0_,
student0.name as name3_0_0_
from
Student student0_
where
student0.id=?
102
Ajeet

```

Mumbai

Data fetched..!

Hibernate:

```

update
Student
set
city=?,
name=?
where
id=?
102
Ajeet

```

Delete operation

```
package com.model;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;

public class Test_get {
    public static void main(String[] args) {

        //configuration
        Configuration cfg=new Configuration()
            .configure("hibernate.cfg.xml");
        SessionFactory sf = cfg.buildSessionFactory();
        Session session = sf.openSession();

        Transaction tx=session.beginTransaction();

        //Hibernate methods
        Student s4=new Student(104,"kartik","Surat");
        session.save(s4);

        System.out.println("data saved");

        //delete
        session.delete(s4);
        tx.commit();
    }
}
```



```
        System.out.println("Data Deleted..!");

        //close the session
        session.close();

    }
}
```

data saved
Hibernate:
insert
into
Student
(city, name, id)
values
(?, ?, ?)
Hibernate:
delete
from
Student
where
id=?
Data Deleted..!

What is Session Factory(I)?

- It is an **Interface**.
- We will get **Session(I) object** from SessionFactory(I)

What is a Session(I)?

- It is an **Interface** which contains the methods like→
- **get(), load(), update(), delete() etc.**

What is Transaction(I) ?

- We need Transaction for **save/update/delete** operation, **not in get** operation.
- We will get Transaction object from **session object**.

Chapter 3

Hibernate/Persistence Object Life cycle

- We can save the object in the DB as a row using Hibernate.

There are **4 states** of the Session object.

1. **Transient state**
2. **Persistent State**
3. **Detached State**
4. **Removed State**

Transient State

- When the object is not associated with **Session** and **Database**. Then that is a Transient state of object.
- In transient state we just create the object and initialize it.

Persistent State

- When the object is associated with **Session** and **Database**. Then that is a Transient state of object.

- When we call the **session.save(object)** method, then object will be the part of persistent state.

Detached State

- When the object is not associated with **Session but it is the part of Database**. Then that is a Transient state of object.
- When we call **session.close()** then object will be the part of detached state.

Removed State

- When the object is associated with **Session but it is not the part of Database**. Then that is a Transient state of object.
- When we call **session.delete(object)** method, then object will be the part of removed state.

